AuraMart — Step-by-Step Build Guide

A learning roadmap for building the AuraMart backend as professional Spring Boot microservices, connected to the existing React frontend. It follows **AuraMart_System_Design.pdf**: the target is **10,000 RPS at peak, p99 < 300 ms, 99.9% availability, zero lost orders**. Steps marked **[SD §n]** implement section *n* of the system design.

How to use this guide - Do the phases in order. Each one builds on the one before. - For every step: **build it → test it → commit it → write down what you learned.** - Every phase ends with a "*You now understand*" checklist. Don't move on until you can explain each point aloud without notes. - Rough total: 14–17 weeks at 2–3 hours a day.

Phase 0 — Professional Setup (Week 1)

#	Step	Concept learned
0.1	Install JDK 21, Maven, Docker Desktop, IntelliJ, Postman, Node 20+	Toolchain
0.2	Create the `auramart` GitHub repo. Protect `main` and work on feature branches (`feat/catalog-crud`)	Git flow, PRs
0.3	Use Conventional Commits (`feat:`, `fix:`, `test:`)	Professional history
0.4	Create a parent `pom.xml` (Java 21, Spring Boot 3.x, Spring Cloud BOM) with modules	Maven multi-module, dependency management
0.5	Add `.editorconfig`, `.gitignore`, and a README with a "How to run" section	Repo hygiene
0.6	Write `docs/adr/0001-microservices.md`: why microservices, and the trade-offs	Architecture Decision Records
0.7	Move the React app into `frontend/` and delete the duplicate copy	Monorepo layout
0.8	Copy the NFR table [SD §2.2] and capacity numbers [SD §3] into `docs/` — every later phase is tested against them	Designing to targets
0.9	Turn on virtual threads in every service: `spring.threads.virtual.enabled=true` [SD §10]	Java 21 concurrency

You now understand: multi-module Maven, BOMs, branching strategy, ADRs.

Phase 1 — Infrastructure with Docker Compose (Week 1)

#	Step	Concept learned
1.1	Write `infra/docker-compose.yml` with MySQL, PostgreSQL (pgvector image), MongoDB, Redis	Containers, volumes, networks
1.2	Add healthchecks, plus `depends_on: condition: service_healthy`	Startup ordering
1.3	Put secrets in a `.env` file (git-ignored) and commit a `.env.example`	12-factor config
1.4	Run `docker compose up -d` and connect to each DB with a client	Debugging containers
1.5	Add a Postgres **read replica** (bitnami/postgresql replication) and a MySQL replica [SD §7.3]	Primary/replica replication
1.6	Add **PgBouncer** in front of Postgres [SD §7.3]	Connection pooling at scale

You now understand: Docker images vs containers, port mapping, volumes, and why each service has its own DB.

Phase 2 — common-lib (Week 2)

#	Step	Concept learned
2.1	Create the `common-lib` module	Shared code (kept small)
2.2	Write a `GlobalExceptionHandler` that returns RFC 9457 `ProblemDetail` with `type: https://auramart.dev/problems/<slug>`	Consistent error contract
2.3	Create a `BusinessException(slug, status)` hierarchy	Exception design
2.4	Write a `RequestIdFilter` that reads or creates `X-Request-Id` and puts it in the MDC	Request tracing basics
2.5	Add shared event DTOs as Java `record`s	Immutability, Java 17+ features

You now understand: `@RestControllerAdvice`, MDC logging, why shared libs should stay small.

Phase 3 — catalog-service (Weeks 2–3) ⭐ the core CRUD skills

#	Step	Concept learned
3.1	Generate it with Spring Initializr: Web, JPA, Validation, PostgreSQL, Flyway, Actuator	Starters
3.2	Write `V1__create_products.sql` and add seed data in `V2__`	Flyway versioned migrations
3.3	Create a `Product` entity with UUID id, `@Version`, `createdAt/updatedAt`, `active`	JPA mapping, auditing
3.4	Layer the code: `controller → service → repository`, with DTOs plus a mapper (MapStruct)	Layered architecture, never expose entities
3.5	Build `GET /products` with `Pageable` and whitelisted sort fields	Pagination, `PagedModel`
3.6	Validate `ProductRequest` with `@Valid` and Bean Validation	Input validation → 400 `validation-error`
3.7	Add `POST/PUT/DELETE`: soft delete, 409 `duplicate-sku`, 409 `concurrent-update` using `expectedUpdatedAt`	Optimistic locking, idempotency of PUT
3.8	Add typo-tolerant search with the `pg_trgm` extension and `similarity()`	Native queries, indexes
3.9	Write **unit tests** with JUnit 5 + Mockito for the service layer	Mocking, AAA pattern
3.10	Write **integration tests** with Testcontainers PostgreSQL and `@SpringBootTest` + MockMvc	Real-DB testing
3.11	Add Swagger UI with springdoc-openapi	API documentation
3.12	Point the frontend at it: `VITE_USE_MOCKS=false`, and browse products for real	Contract-first integration
3.13	Route reads to the replica: `AbstractRoutingDataSource` + `@Transactional(readOnly = true)` [SD §6.1]	Read/write splitting
3.14	Create the indexes from [SD §7.2] (`category, active, price`; GIN trigram) and verify with `EXPLAIN ANALYZE`	Query tuning
3.15	Tune HikariCP: `maximum-pool-size=20`, timeouts	Pool sizing
3.16	Seed **1 M products** with a script so you test at real data size [SD §3.2]	Realistic data

You now understand: JPA/Hibernate lifecycle, N+1 queries (fix them with `@EntityGraph` / fetch joins), transactions, pagination, validation, testing pyramid.

Phase 4 — Redis Caching + S3 Images (Week 4)

#	Step	Concept learned
4.1	Add `spring-boot-starter-data-redis` and turn on `@EnableCaching`	Cache abstraction
4.2	Use `@Cacheable("product")` on get-by-id, and `@CacheEvict` on update/delete	Cache-aside, invalidation
4.3	Configure TTL and a JSON serializer	Serialization pitfalls
4.4	Measure response time before and after in Postman	Proving performance gains
4.5	Create an AWS free-tier account and an S3 bucket. Use an IAM user with an S3-only policy	AWS IAM, least privilege
4.6	Upload product images with the AWS SDK v2 and return **pre-signed URLs**	Object storage
4.7	Use LocalStack in Docker for local S3	Dev/prod parity
4.8	Cache list pages `products:list:{hash(params)}` and search results, TTL 10 min + **random jitter** [SD §8]	Cache avalanche
4.9	Prevent **stampede**: single-flight lock with `SET NX` on miss	Cache stampede
4.10	Prevent **penetration**: cache "not found" for 60 s	Cache penetration
4.11	Evict on the `ProductUpdated` event (added in Phase 7) so every pod is consistent	Event-driven invalidation
4.12	Add a Caffeine local cache for categories (60 s)	Two-level caching
4.13	Expose the cache hit rate as a metric; target **> 90%** [SD §13]	Measuring caches

You now understand: when to cache and when not to, TTLs, stale data, IAM policies, S3.

Phase 5 — auth-service + API Gateway (Weeks 5–6) 🔐

#	Step	Concept learned
5.1	auth-service: `User`, `Role` tables in MySQL (Flyway), BCrypt passwords	Password hashing
5.2	`POST /auth/register`, `POST /auth/login` → access JWT (15 min) + refresh token (7 days)	JWT structure, claims, expiry
5.3	Configure `SecurityFilterChain`, stateless sessions, method security with `@PreAuthorize("hasRole('ADMIN')")`	Spring Security internals
5.4	Add `POST /auth/refresh` and logout (store a refresh-token blacklist in Redis)	Token rotation
5.5	api-gateway: Spring Cloud Gateway routes for each `/api/v1/**` path	Reverse proxy, routing
5.6	Add a global filter that validates the JWT and injects `X-Customer-Id` and `X-Roles` headers	Edge security
5.7	Configure CORS at the gateway only	CORS
5.8	Add Redis `RequestRateLimiter` (e.g. 20 req/s per user)	Rate limiting, token bucket
5.9	Sign JWTs with **RS256**; the gateway verifies them with the public key only [SD §12]	Asymmetric keys
5.10	Use separate limits: 100 req/s per IP (anonymous), 50 req/s per user, **10 AI msgs/min** [SD §12]	Tiered rate limits

#	Step	Concept learned
5.11	Make the rate limiter **fail open** if Redis is down [SD §11]	Availability over strictness
5.12	(Optional) Swap in Keycloak OIDC — the frontend already supports `VITE_AUTH_MODE=oidc`	OAuth2 / OIDC, PKCE

You now understand: authentication vs authorization, JWT vs session, filters, the API Gateway pattern, CORS.

Phase 6 — inventory-service + order-service with Feign (Weeks 6–7)

#	Step	Concept learned
6.1	inventory-service: `Stock(productId, available, reserved, @Version)`	Separate bounded context
6.2	Write a concurrency test: 50 threads buy the last item and only 1 succeeds	`OptimisticLockException`, retries
6.3	order-service: `Cart`, `CartItem` in MySQL; `GET/POST/PATCH/DELETE /carts/current/**`	Aggregate design
6.4	Call catalog (price) and inventory (stock) with **OpenFeign**	Declarative HTTP clients
6.5	Add Resilience4j: timeout, retry, circuit breaker, fallback	Fault tolerance
6.6	Add a Feign `ErrorDecoder` that maps remote 404/409 to local exceptions	Error propagation
6.7	Add service discovery with Eureka (or Docker DNS names)	Service discovery
6.8	Store the **cart in Redis** (`HSET cart:{customerId}`, TTL 30 d) with async write-behind to MySQL [SD §6.2]	Choosing the right store
6.9	Use the exact Resilience4j values from [SD §11]: timeout 300 ms, 2 retries with jitter, breaker 50% / 20 calls	Tuning resilience
6.10	Make the stock check on add-to-cart **soft only** — reserve at checkout	Where consistency matters

You now understand: synchronous inter-service calls, cascading failures, circuit breakers, race conditions.

Phase 7 — Kafka Checkout Saga (Weeks 8–9) ⭐ the hardest and most valuable

#	Step	Concept learned
7.1	Add Kafka (KRaft) and Kafka UI to compose. Create topics with 3 partitions	Topics, partitions, consumer groups
7.2	`POST /orders`: save the order as `PENDING` **and** an `outbox` row in the same transaction	**Transactional outbox** (dual-write problem)
7.3	Add an outbox relay (`@Scheduled` poller) that publishes to `order.events` with key `orderId`	Ordering per key
7.4	inventory consumes `OrderPlaced` and publishes `StockReserved` or `StockRejected`	Event-driven design
7.5	payment-service (simulated, ~20% failures) publishes `PaymentSucceeded` or `PaymentFailed`	Choreography
7.6	order-service moves to `CONFIRMED` or `CANCELLED`. On payment failure, inventory **releases** stock	**Saga + compensation**
7.7	Store processed `eventId`s for **idempotent consumers**	At-least-once delivery

#	Step	Concept learned
7.8	Write Testcontainers Kafka integration tests for the happy path and both failure paths	Async testing (Awaitility)
7.9	Watch the frontend poll the order from PENDING to CONFIRMED	End-to-end
7.10	Create topics with the partition counts from [SD §9]; producers `acks=all`, `enable.idempotence=true`; `min.insync.replicas=2`	Durable Kafka config
7.11	Require an `Idempotency-Key` header on `POST /orders` (unique column) [SD §6.3]	Safe client retries
7.12	Write the conditional stock update `WHERE available - reserved >= :q AND version = :v` [SD §6.3]	Atomic reservation
7.13	Re-key reservations by **productId** onto `inventory.reservations` (24 partitions) [SD §6.3]	Hot-key serialization
7.14	Add a **saga timeout**: a scheduler cancels orders PENDING > 60 s and releases stock [SD §11]	Stuck-saga recovery
7.15	(Bonus) Replace the poller with **Debezium CDC** for the outbox	Change data capture
7.16	Partition the `orders` table by month [SD §7.3]	Table partitioning

You now understand: why 2-phase commit is avoided, eventual consistency, exactly-once vs at-least-once, outbox, saga.

Phase 8 — notification-service (Week 10)

#	Step	Concept learned
8.1	Consume `OrderConfirmed` / `OrderCancelled`	Fan-out consumers
8.2	Store a notification log in **MongoDB** with Spring Data MongoDB	Document DB vs relational
8.3	Send emails with JavaMailSender + Thymeleaf templates; use MailHog locally	Email delivery
8.4	Add `@RetryableTopic(attempts=3, backoff=exponential)` and a `@DltHandler`	Retries, Dead Letter Topic
8.5	Force a failure and watch the message reach the DLT in Kafka UI	Failure handling

You now understand: when to choose MongoDB, poison messages, non-blocking retries.

Phase 9 — AI Assistant: RAG + Tools + SSE (Weeks 11–12) 🤖

#	Step	Concept learned
9.1	Add Spring AI with Gemini (chat + embedding models); keep the API key in env vars	LLM integration
9.2	Consume `product.events` → build text → embed → store in **pgvector** (HNSW index)	Embeddings, vector search
9.3	Add `GET /products/semantic-search` with cosine similarity and a score threshold	Semantic search
9.4	`POST /assistant/chat`: retrieve the top-k products, put them in the prompt, answer only from context	**RAG**, grounding
9.5	Add `@Tool searchProducts` and `@Tool getOrderStatus` (calls order-service through Feign)	**LLM tool calling**
9.6	Store conversation memory per `conversationId` in Redis	Chat memory
9.7	Add `POST /assistant/chat/stream` returning `Flux<ServerSentEvent>` with `meta/tool/token/done` events	**SSE streaming**, WebFlux basics
9.8	Turn off gateway buffering for SSE	Streaming through proxies

#	Step	Concept learned
9.9	Write a prompt-injection test ("ignore instructions…") and handle it	AI safety
9.10	Add a **bulkhead** (max ~25 concurrent LLM streams per pod), a 10 s timeout and a fallback message [SD §6.4]	Isolating slow dependencies
9.11	Tune HNSW (`ef_search=40`) and measure recall vs latency on 1 M vectors	Vector index tuning
9.12	Cache embeddings of frequent queries in Redis	Cost control
9.13	Tools run **as the calling user** only (an order lookup can't read someone else's order) [SD §12]	AI authorization

You now understand: embeddings, chunking, vector indexes, RAG vs fine-tuning, tool calling, streaming.

Phase 10 — Observability (Week 13)

#	Step	Concept learned
10.1	Log as JSON with Logstash encoder, including `requestId`, `traceId`, `service`	Structured logging
10.2	Add Elasticsearch + **Kibana** (or Filebeat) to compose; search one request across services	Centralized logs
10.3	Add Micrometer Tracing + OpenTelemetry → Tempo/Zipkin	Distributed tracing
10.4	Add Actuator + Prometheus + a Grafana dashboard (RPS, latency, errors)	Metrics, RED method
10.5	Add custom metrics: `orders.confirmed`, `orders.cancelled`	Business metrics
10.6	Define the **SLOs** from [SD §13] and set up Alertmanager rules: 5xx > 0.5%, p99 > 500 ms, Kafka lag > 5,000, any DLT message	SLOs and alerting
10.7	Build a Grafana dashboard for Kafka consumer lag and Hikari pool usage	Spotting bottlenecks
10.8	Sample traces (e.g. 10%) and keep logs for 7 days, because 40 MB/s of logs is expensive [SD §3.2]	Observability cost

You now understand: the three pillars of observability: logs, metrics and traces.

Phase 11 — Docker, CI/CD, AWS Deploy (Weeks 14–15) 🚀

#	Step	Concept learned
11.1	Write a multi-stage Dockerfile per service (or use `spring-boot:build-image`); run as non-root	Small, secure images
11.2	Write a full `docker-compose.yml` that starts **everything** with one command	Orchestration
11.3	Add GitHub Actions: `mvn verify` → `npm test` → build images → push to Docker Hub / ECR	CI pipeline
11.4	Add quality gates: JaCoCo coverage ≥ 70%, Checkstyle/Spotless, dependency scan	Code quality
11.5	Launch an AWS EC2 instance (t3.medium) with an IAM **role** for S3 (no keys on the box)	EC2, IAM roles
11.6	CD: on merge to `main`, SSH-deploy with `docker compose pull && up -d`	Continuous deployment
11.7	Add Nginx + Let's Encrypt HTTPS in front of the gateway	TLS, reverse proxy
11.8	Add Kubernetes manifests + HPA with kind/minikube locally (CPU 60%, min 2 replicas) [SD §10]	K8s, autoscaling

#	Step	Concept learned
11.9	Add readiness/liveness probes and graceful shutdown (`server.shutdown=graceful`)	Zero-downtime deploys
11.10	Serve the React app and images from **CloudFront**; cache anonymous `GET /products` for 30 s [SD §8]	CDN offload
11.11	Add **AWS WAF** managed rules in front of the ALB [SD §12]	Edge protection
11.12	Move secrets to **AWS Secrets Manager**	Secret management
11.13	(Bonus) Write Terraform for the VPC across 3 AZs [SD §14]	Infrastructure as code
11.14	Add a canary deploy (10% → 100%) with automatic rollback when the SLOs fail [SD §14]	Progressive delivery

You now understand: the full path from commit to production.

Phase 11.5 — Prove 10,000 RPS (Week 15–16) 📈 [SD §15]

Do this phase on a staging setup sized like [SD §3.4], or scale everything down proportionally on one machine (e.g. a 1,000 RPS target on a laptop).

#	Step	Concept learned
L.1	Write k6 scripts that replay the **traffic mix** from [SD §3.1]: 55% browse, 15% search, 17% cart, etc.	Realistic load models
L.2	**Baseline**: 1k RPS for 10 min → record p50/p99, CPU and DB load	Benchmarking
L.3	**Find the bottleneck**: raise load until something breaks (pool exhaustion, GC, DB CPU, Kafka lag). Fix it, then repeat	Performance engineering loop
L.4	**Peak**: ramp to 10k RPS over 5 min and hold for 15 min. Pass: p99 < 300 ms, errors < 0.1%	Capacity verification
L.5	**Spike**: 2k → 15k RPS in 30 s. The HPA scales, the rate limiter returns 429s, and there are no 5xx storms	Elasticity
L.6	**Flash sale**: 500 orders/s on 1 SKU with 100 units. Pass: **exactly 100 confirmed**	Overselling proof
L.7	**Soak**: 3k RPS for 8 h. No memory leaks (check heap and GC graphs)	Stability
L.8	**Chaos**: kill a pod, a Kafka broker and the Redis primary during load. SLOs hold and no orders are lost	Resilience proof
L.9	Profile with JFR / async-profiler and fix the top 3 hot spots	JVM profiling
L.10	Write `docs/performance-report.md` with graphs: before/after cache, before/after replicas	Showing results

You now understand: how to *prove* scale with numbers, rather than just claim it — this is what senior interviews ask.

Phase 12 — Professional Polish (Week 17)

#	Step	Concept learned
12.1	Add a Postman collection plus environment in `docs/`	API testing
12.2	Add a k6 or JMeter load test on `GET /products` with and without cache	Performance testing
12.3	Update the README with the architecture diagram, how to run, screenshots and a demo GIF	Presentation
12.4	Write ADRs for Kafka vs RabbitMQ, outbox vs CDC, and choreography vs orchestration	Trade-off thinking

#	Step	Concept learned
12.5	Record a 5-minute demo video of the interview scenario	Communication
12.6	Write 2–3 Hashnode articles (outbox pattern, RAG with pgvector)	Teaching = mastery

Professional Rules (follow in every phase)

1. **One feature = one branch = one PR.** Review your own diff before merging.
2. **Tests before merge.** Unit tests for logic; Testcontainers for DB and Kafka.
3. **Never expose entities** — always DTOs.
4. **No secrets in Git.** Use env vars, `.env.example` and AWS IAM roles.
5. **Every service has a database of its own.** No cross-service joins.
6. **Every endpoint is in Swagger** and matches `API_CONTRACT.md`.
7. **Log with context** (`requestId`, `orderId`), never `System.out`.
8. **Write a learning note** after each phase: what broke, why, and how you fixed it.

Interview Questions You'll Be Able to Answer

- How do you handle a distributed transaction without 2PC? → *Saga + outbox*
- How do you prevent overselling? → *Optimistic locking with `@Version`*
- What if Kafka delivers a message twice? → *Idempotent consumer*
- How do you keep a cache consistent? → *Cache-aside + evict on write + TTL*
- What happens if a downstream service is slow? → *Timeout + circuit breaker + fallback*
- How does your AI avoid hallucinating products? → *RAG grounded on pgvector results*
- How do you trace one request across 8 services? → *OpenTelemetry trace ID + Kibana*
- How did you size the system for 10k RPS? → *Traffic mix → per-pod throughput → pods + 50% headroom; cache cuts DB reads 20×*
- What about a flash sale on one product? → *Re-key by productId + atomic conditional update; load-tested exactly 100/100*
- What breaks first under load? → *Measured it: DB pool, then cache misses — fixed with replicas and PgBouncer*
- What do you turn off when overloaded? → *Shed AI first, protect checkout last*
- How do you secure microservices? → *JWT validated at the gateway, roles in the services*

Final Checklist

- [ ] `docker compose up` starts the whole system
- [ ] Frontend works with `VITE_USE_MOCKS=false`
- [ ] Checkout saga: success, out-of-stock, and payment-failure paths work
- [ ] AI assistant streams grounded answers and looks up orders
- [ ] CI is green, with coverage ≥ 70%
- [ ] Deployed on AWS EC2 with HTTPS
- [ ] k6 peak test passes the targets (p99 < 300 ms, errors < 0.1%), with the performance report committed
- [ ] Flash-sale test: exactly 100 of 100 units sold, no overselling
- [ ] Chaos test: no orders lost when a pod, broker or Redis node is killed
- [ ] SLO alerts fire correctly in Grafana
- [ ] README, ADRs, Postman collection, demo video done